

FICHE TECHNIQUE**Correction Ollama v0.30.x — Activation Vulkan / iGPU**

Nœuds d'inférence locale lAbraïn — mini-PC AMD (LX15 Pro, Geekom A7 Max) et portable Intel (Lenovo IdeaPad, Iris Xe)

ADRASEC 77 · FNRASEC Réf. PB_Ollama_v30 Ollama 0.30.7 12 juin 2026 F1GBD

1. Objet

Cette fiche décrit les corrections à appliquer après la mise à jour vers Ollama v0.30.x sur les nœuds d'inférence locale. Deux anomalies apparaissent à la suite de cette montée de version :

- une erreur 500 qui bloque tout chargement de modèle ;
- l'absence d'utilisation du GPU : l'inférence tourne à 100 % sur le CPU.

Les deux corrections sont indépendantes : la première débloque le chargement, la seconde bascule l'inférence sur le GPU intégré (iGPU) via le backend Vulkan.

2. Symptômes

2.1 — Erreur 500 au chargement du modèle

Tout appel se solde par une terminaison du process llama-server (code 0xc0000005) :

```
ollama run qwen2.5:7b "test"
Error: 500 Internal Server Error: llama-server process has
terminated: exit status 0xc0000005 ...
llama_init_from_model: V cache quantization requires flash_attn
```

2.2 — GPU non utilisé (100 % CPU)

Une fois l'erreur 500 corrigée, le modèle se charge mais reste intégralement sur le CPU :

```
ollama ps
NAME          ID          SIZE    PROCESSOR  CONTEXT
qwen2.5:7b    845dbda0ea48  5.2 GB  100% CPU   4096
```

3. Causes

3.1 — Conflit cache KV quantifié / Flash Attention

Le profil ADRASEC active la quantification du cache KV (OLLAMA_KV_CACHE_TYPE=q8_0) tout en gardant Flash Attention désactivé (OLLAMA_FLASH_ATTENTION=0), choix retenu pour les charges RAG sur iGPU. Or la quantification du cache des valeurs (V) exige Flash Attention. Jusqu'à la v0.29, Ollama ignorait silencieusement ce conflit (repli automatique en f16) ; la v0.30.x ne tolère plus la combinaison et fait planter le runner.

3.2 — Variables GPU absentes ou non persistées

L'inférence reste sur CPU tant que OLLAMA_VULKAN et, pour les GPU intégrés, OLLAMA_IGPU_ENABLE ne sont pas déclarés et chargés par le serveur au démarrage. Piège fréquent : une variable posée uniquement pour la session courante (préfixe \$env: en

PowerShell) est perdue dès le redémarrage suivant du serveur — le GPU « marche » puis « disparaît » au test suivant.

4. Correction — Étape 1 : débloquer le chargement (cache KV)

Forcer le cache KV en f16 au niveau Machine (scope vu par tous les comptes, y compris un démarrage automatique), puis redémarrer le poste. Le f16 n'a aucune dépendance à Flash Attention : la convention RAG « Flash Attention désactivé » est préservée.

Dans une console PowerShell Administrateur :

```
# Console PowerShell ADMINISTRATEUR (scope Machine)
[Environment]::SetEnvironmentVariable("OLLAMA_KV_CACHE_TYPE","f16","Machine")

# Puis redémarrer le PC pour appliquer
```

Vérification après reboot : le chargement ne renvoie plus d'erreur 500.

Variante : supprimer purement la variable produit le même effet, f16 étant la valeur par défaut.

5. Correction — Étape 2 : activer le GPU (Vulkan + iGPU)

Déclarer les deux variables d'activation. Pour les GPU intégrés — aussi bien les APU AMD que les Intel Iris Xe — les deux sont nécessaires : OLLAMA_VULKAN force le backend Vulkan, OLLAMA_IGPU_ENABLE autorise l'usage d'un GPU intégré.

```
# Déclaration des variables (usage interactif : scope User)
[Environment]::SetEnvironmentVariable("OLLAMA_VULKAN","1","User")
[Environment]::SetEnvironmentVariable("OLLAMA_IGPU_ENABLE","1","User")

# Vérification (doit renvoyer 1 et 1)
[Environment]::GetEnvironmentVariable("OLLAMA_VULKAN","User")
[Environment]::GetEnvironmentVariable("OLLAMA_IGPU_ENABLE","User")
```

Redémarrer ensuite le serveur Ollama pour qu'il hérite des variables : fermer l'application (icône tray), arrêter tous les process, puis relancer depuis une nouvelle fenêtre — ou rebooter.

```
# Arrêt complet de tous les serveurs Ollama
Get-Process -EA SilentlyContinue ollama, 'ollama app',
ollama_llama_server, llama-server | Stop-Process -Force
```

Important : ouvrir une nouvelle console après la pose des variables — une console déjà ouverte conserve l'ancien environnement.

6. Spécificités par type de machine

Machine	GPU	Backend	Points d'attention
LX15 Pro	AMD Radeon (iGPU)	Vulkan	VULKAN=1 + IGPU_ENABLE=1 + KV=f16. Flash Attention laissé désactivé (RAG).
Geekom A7 Max	AMD Radeon (iGPU)	Vulkan	Mêmes variables que le LX15 Pro. Détection Vulkan fiable.
Lenovo IdeaPad Flex 5 (i5-1135G7)	Intel Iris Xe	Vulkan	Mêmes variables. Pilote Intel Graphics et runtime Vulkan à jour

Machine	GPU	Backend	Points d'attention
			requis ; détection Iris Xe plus capricieuse.

Cas Intel Iris Xe : si, malgré les deux variables, Ollama ne charge pas le backend Vulkan (log sans ligne « inference compute ... library=Vulkan », VRAM totale à 0), vérifier d'abord le pilote Intel et le runtime Vulkan (vulkaninfo doit lister l'Iris Xe). En dernier recours, recourir à la build IPEX-LLM d'Ollama (backend SYCL/oneAPI Intel), en forçant l'offload complet avec OLLAMA_NUM_GPU=999.

Rappel de dimensionnement : sur un iGPU à mémoire partagée, un modèle 7B reste limité par la bande passante mémoire. Pour un poste type Iris Xe, un modèle 3B/4B offre un bien meilleur compromis réactivité/charge.

7. Portée des variables et démarrage headless

- Scope User : suffisant pour l'usage interactif sous le compte connecté.
- Scope Machine : obligatoire si le serveur démarre via une tâche planifiée / service sous un autre compte (cold-boot autonome, watchdog). Poser alors les trois variables en Machine.
- Préfixe \$env: : valable uniquement pour la console courante — jamais persistant.

Variable	Valeur	Scope conseillé	Rôle
OLLAMA_KV_CACHE_TYPE	f16	Machine	Évite le conflit cache V / Flash Attention (erreur 500).
OLLAMA_FLASH_ATTENTION	0 (false)	laisser désactivé	Convention RAG : Flash Attention off.
OLLAMA_VULKAN	1	User (Machine si headless)	Force le backend Vulkan.
OLLAMA_IGPU_ENABLE	1	User (Machine si headless)	Autorise l'usage du GPU intégré.

8. Procédure de test final

1. Version : `ollama --version` → doit indiquer 0.30.7 ou supérieur.
2. Démarrer le serveur au premier plan (`ollama serve`) et contrôler la ligne « server config » du log : elle doit afficher OLLAMA_KV_CACHE_TYPE:f16, OLLAMA_FLASH_ATTENTION:false, OLLAMA_VULKAN:true, OLLAMA_IGPU_ENABLE:1.
3. Contrôler la découverte GPU dans le log : une ligne `inference compute ... library=Vulkan ... type=iGPU` avec une VRAM totale non nulle (ex. « 16.0 GiB »).
4. Charger un modèle : `ollama run qwen2.5:7b "test"` → la réponse doit être cohérente (pas de charabia, signe d'un offload Vulkan défaillant).
5. Vérifier le processeur : `ollama ps` → la colonne PROCESSOR doit afficher 100% GPU (ou un fort pourcentage GPU).
6. Test de persistance : redémarrer la machine, refaire les points 4 et 5 **sans rien retoucher** → le GPU doit rester actif.

Résultat attendu :

```
ollama ps
NAME          ID          SIZE    PROCESSOR  CONTEXT
qwen2.5:7b    845dbda0ea48 4.9 GB  100% GPU   4096
```

```
# OK : Ollama utilise bien le GPU à 100 %.
```

9. Récapitulatif express

Console PowerShell Administrateur, puis reboot :

```
[Environment]::SetEnvironmentVariable("OLLAMA_KV_CACHE_TYPE","f16","Machine")
[Environment]::SetEnvironmentVariable("OLLAMA_VULKAN","1","Machine")
[Environment]::SetEnvironmentVariable("OLLAMA_IGPU_ENABLE","1","Machine")

# Redémarrer le PC, puis : ollama run qwen2.5:7b "test" / ollama ps
```

Note : en scope Machine, les trois variables sont vues à la fois par la session interactive et par un démarrage automatique — configuration recommandée pour les nœuds autonomes.